plb

Lab Kafka Administration

Visite guidée des interfaces : Kafka UI, Grafana et Prometheus

Formation PLB IKAF · Apache Kafka 4.3.2
Captures réalisées le 28 octobre 2026

Le lab de la formation

Chaque participant dispose sur son poste d'un cluster **Apache Kafka 4.3.2** complet, lancé avec Docker : trois nœuds qui sont à la fois broker et contrôleur KRaft, un poste d'administration en ligne de commande, et trois interfaces web.

Ce document parcourt ces interfaces écran par écran. Sur chaque capture, des repères numérotés renvoient aux explications de la colonne de droite. Un encadré distingue, quand c'est utile, le réglage du lab de la pratique en production.

Interface	Adresse	Rôle	Mode du lab
Kafka UI (kafbat UI 1.5.0)	http://localhost:8080	Voir et agir : brokers, topics, messages, consumer groups, ACL	Tous les modes
Grafana 13.2.4	http://localhost:3000	Suivre dans le temps : un tableau de bord de 27 panneaux en 7 rangées	extended, full
Prometheus 3.15.1	http://localhost:9090	Collecter les métriques toutes les 15 s, évaluer les 9 règles d'alerte	extended, full

1 élément repéré : le numéro renvoie à l'explication de même numéro.

État du lab au moment des captures

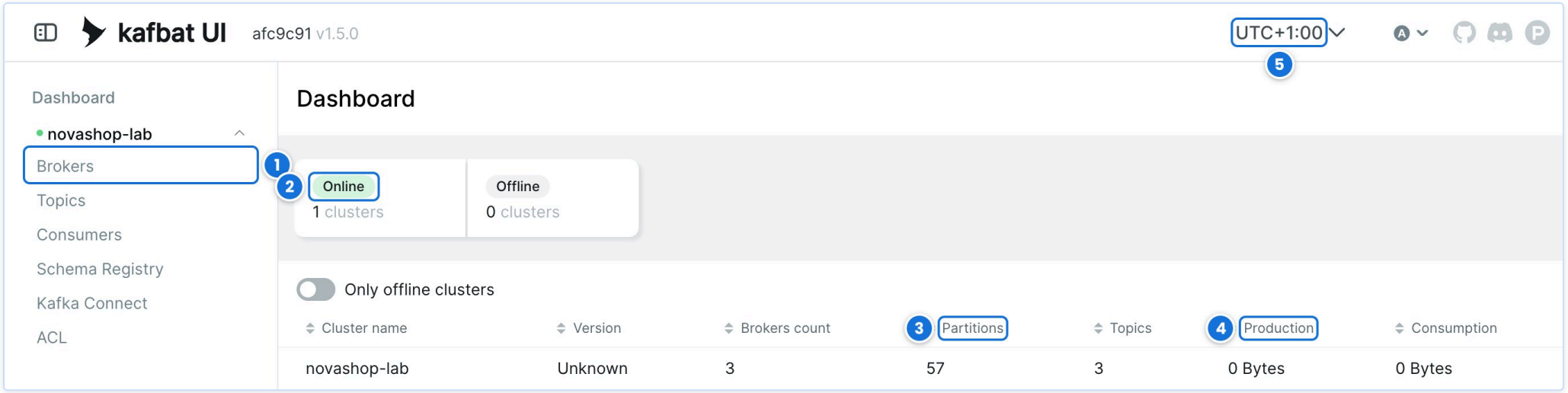
- Mode `extended` : cluster, Kafka UI, Prometheus, Grafana, kafka-exporter et générateur de charge.
- Trois topics : `loadgen` (alimenté en continu à 100 messages par seconde), `lab-validate` (créé par le script de validation) et le topic interne `__consumer_offsets`.
- Un consumer group, `loadgen-consumers`. Aucune ACL. Aucun incident en cours.
- Les valeurs citées (débits, lag, latences) sont celles du lab de référence ; elles varient légèrement d'un poste à l'autre.

Sommaire

Partie	Pages
Kafka UI	3 à 13
Grafana	14 à 17
Prometheus	18 à 20
Lab ou production : l'essentiel	21

La vue d'ensemble

Kafka UI est un client du cluster, comme une application : elle s'y connecte par le listener interne et affiche ce que Kafka lui décrit.



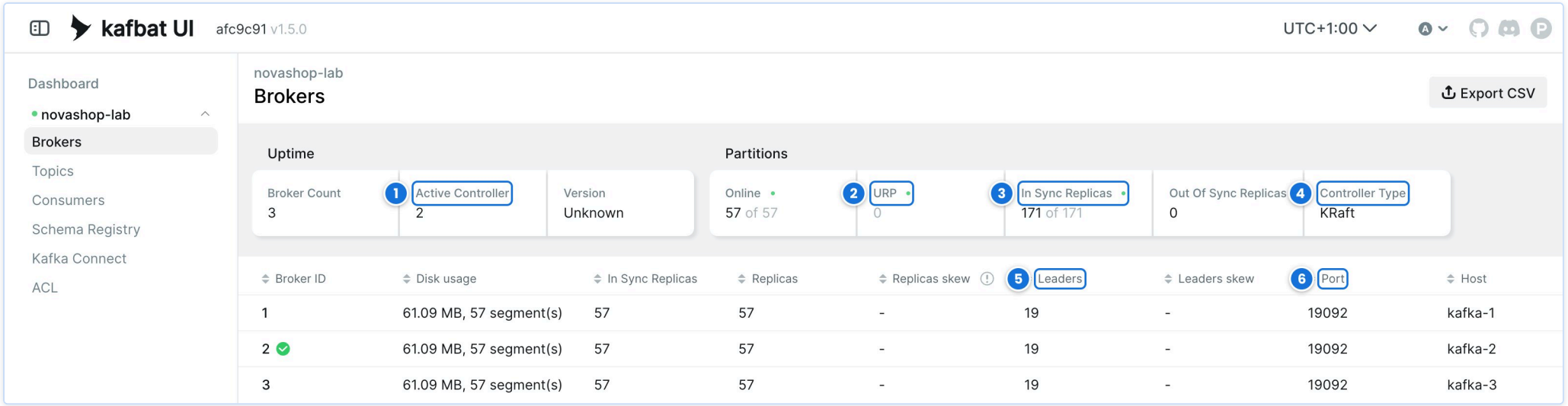
1. Le menu du cluster `novashop-lab` : brokers, topics, consumer groups, Schema Registry, Kafka Connect et ACL.
2. Les clusters déclarés. Une même instance peut en piloter plusieurs ; le lab en a un, en ligne.
3. 3 brokers, 3 topics et 57 partitions : 50 pour le topic interne `__consumer_offsets`, 6 pour `loadgen`, 1 pour `lab-validate`.
4. Version « Unknown » et débits à zéro : dans le lab, Kafka UI ne lit pas les métriques des brokers. Les débits se suivent dans Grafana.
5. Fuseau horaire d'affichage, celui du navigateur. Les traces des conteneurs sont en UTC : en France, une heure d'écart l'hiver, deux l'été.

Lab ou production

Lab : aucune authentification, toutes les actions permises. Production : authentification unique (SSO), rôles par équipe, lecture seule pour la plupart des utilisateurs, et un compte de service dédié aux droits limités.

Brokers : le bilan de santé

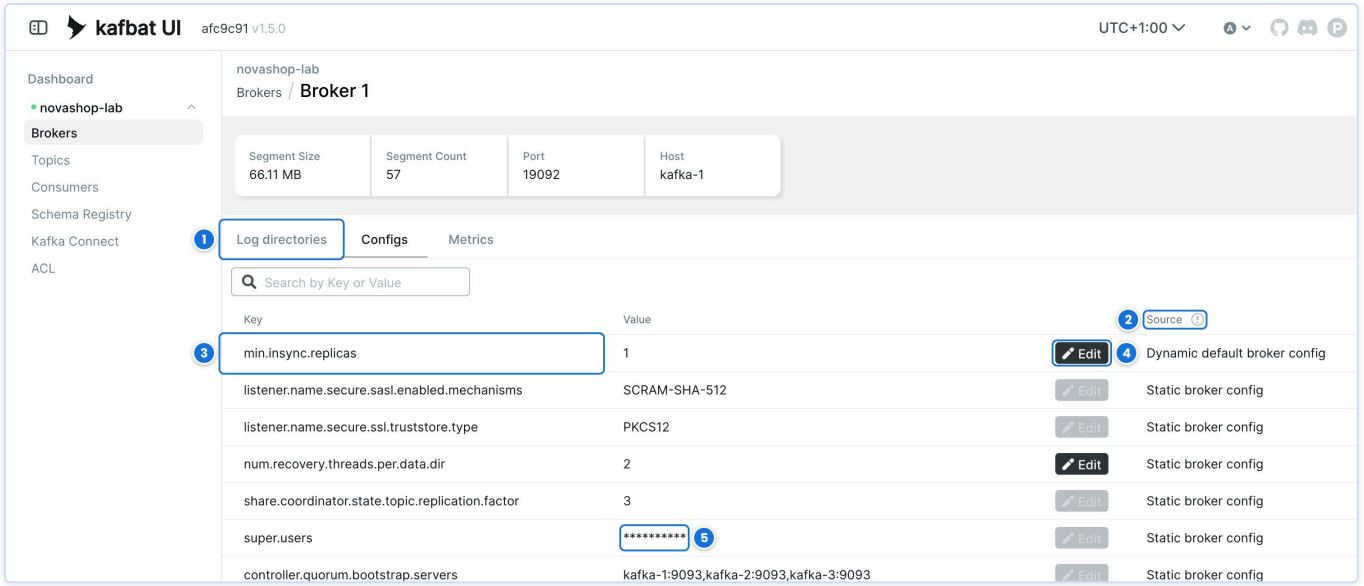
La page à ouvrir en premier : elle dit si toutes les partitions ont leurs copies, et comment la charge se répartit entre les brokers.



1. Le contrôleur actif. En KRaft, c'est le leader du quorum : le nœud qui écrit les métadonnées du cluster. Une coche verte le repère dans le tableau.
2. Partitions sous-répliquées (*under-replicated*) : au moins une copie ne suit plus le leader. Indicateur de santé principal, attendu à 0.
3. Répliques synchronisées : 171 sur 171, soit 57 partitions fois 3 copies.
4. KRaft : le cluster gère ses métadonnées lui-même. ZooKeeper a disparu avec Kafka 4.0.
5. 57 répliques et 19 leaders par broker : la charge est équilibrée. Les colonnes « skew » mesurent l'écart.
6. 19092 : le listener interne, celui par lequel Kafka UI parle aux brokers.

Un broker : sa configuration

Le détail d'un broker présente ses répertoires de données, sa configuration complète et, si elles sont branchées, ses métriques.



1. Trois onglets : répertoires de données (un seul dans le lab ; un second est ajouté en travaux pratiques), configuration, métriques (non branchées dans le lab).

2. L'origine de chaque valeur. *Static broker config* : le fichier `server.properties`, lu au démarrage. *Dynamic* : une valeur posée à chaud, stockée dans les métadonnées du cluster, prioritaire sur le fichier.

3. `min.insync.replicas` vaut 1, posé au niveau du cluster par le contrôleur. Un producteur en `acks=all` peut alors être acquitté par le seul leader. La formation montre comment le porter à 2.

4. Modification à chaud, équivalente à `kafka-configs.sh --alter`.

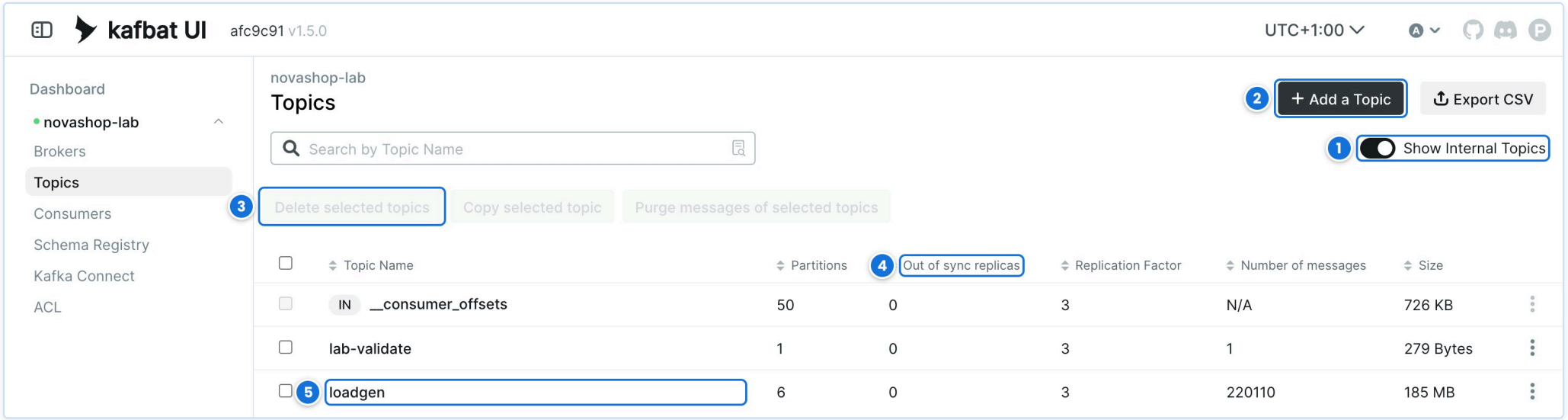
5. Les secrets sont masqués, comme `super.users`.

Lab ou production

Lab : `super.users` contient `User:ANONYMOUS`, pour que les premiers travaux pratiques se passent d'authentification. Jamais en production, où une modification de configuration passe par un changement scripté, relu et tracé.

Topics : la liste

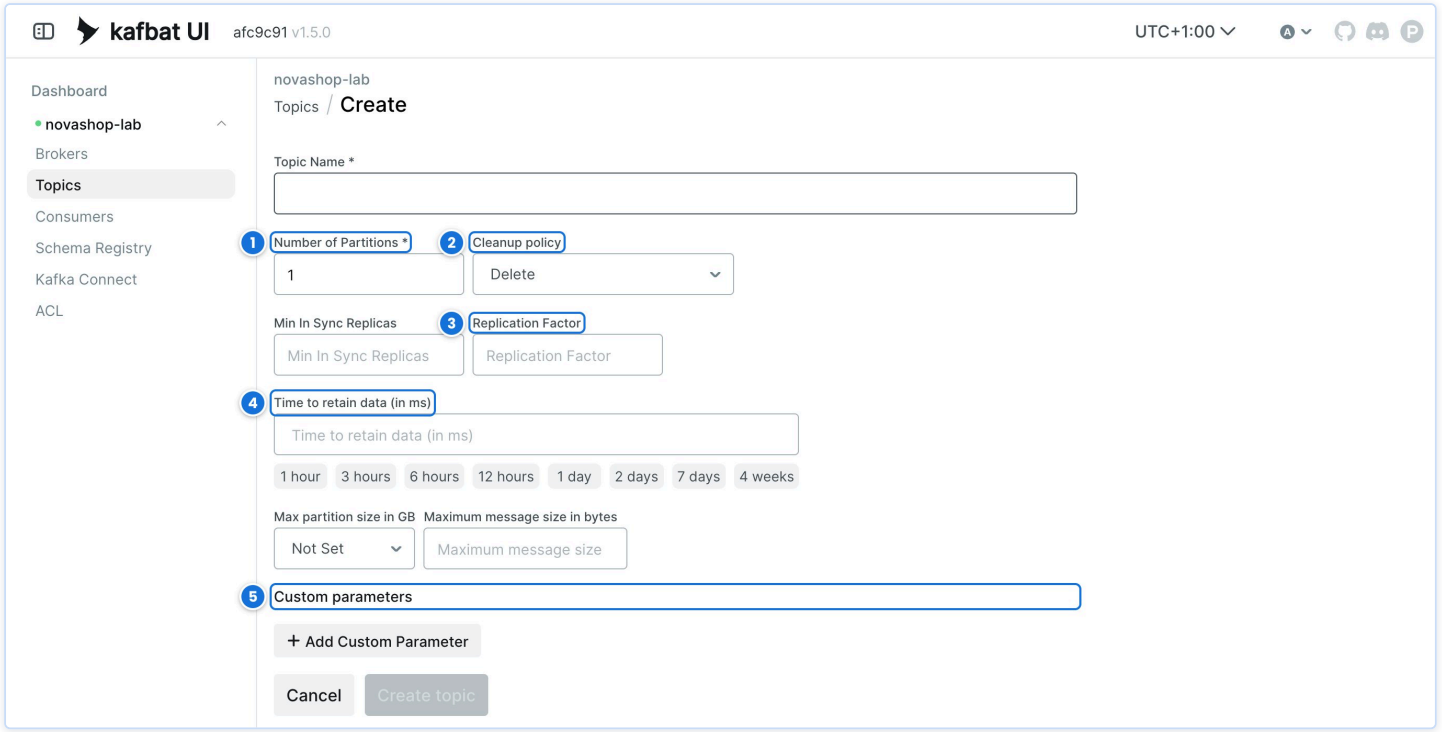
Tous les topics du cluster, avec pour chacun ses partitions, sa réplication, son volume et son nombre de messages.



1. Affiche les topics internes, dont `__consumer_offsets` (pastille IN), où les consumer groups enregistrent leur position. Activé par défaut.
2. Création d'un topic : formulaire présenté page suivante.
3. Actions groupées sur les topics cochés : suppression, copie, purge des messages. Irréversibles.
4. Par topic : partitions, répliques hors ISR, facteur de réplication, nombre de messages et taille.
5. `loadgen` reçoit en continu 100 messages par seconde de 256 octets, envoyés par le générateur de charge du lab.

Créer un topic

Le formulaire reprend les choix faits en ligne de commande avec `kafka-topics.sh --create`. Les champs laissés vides prennent la valeur par défaut du cluster.



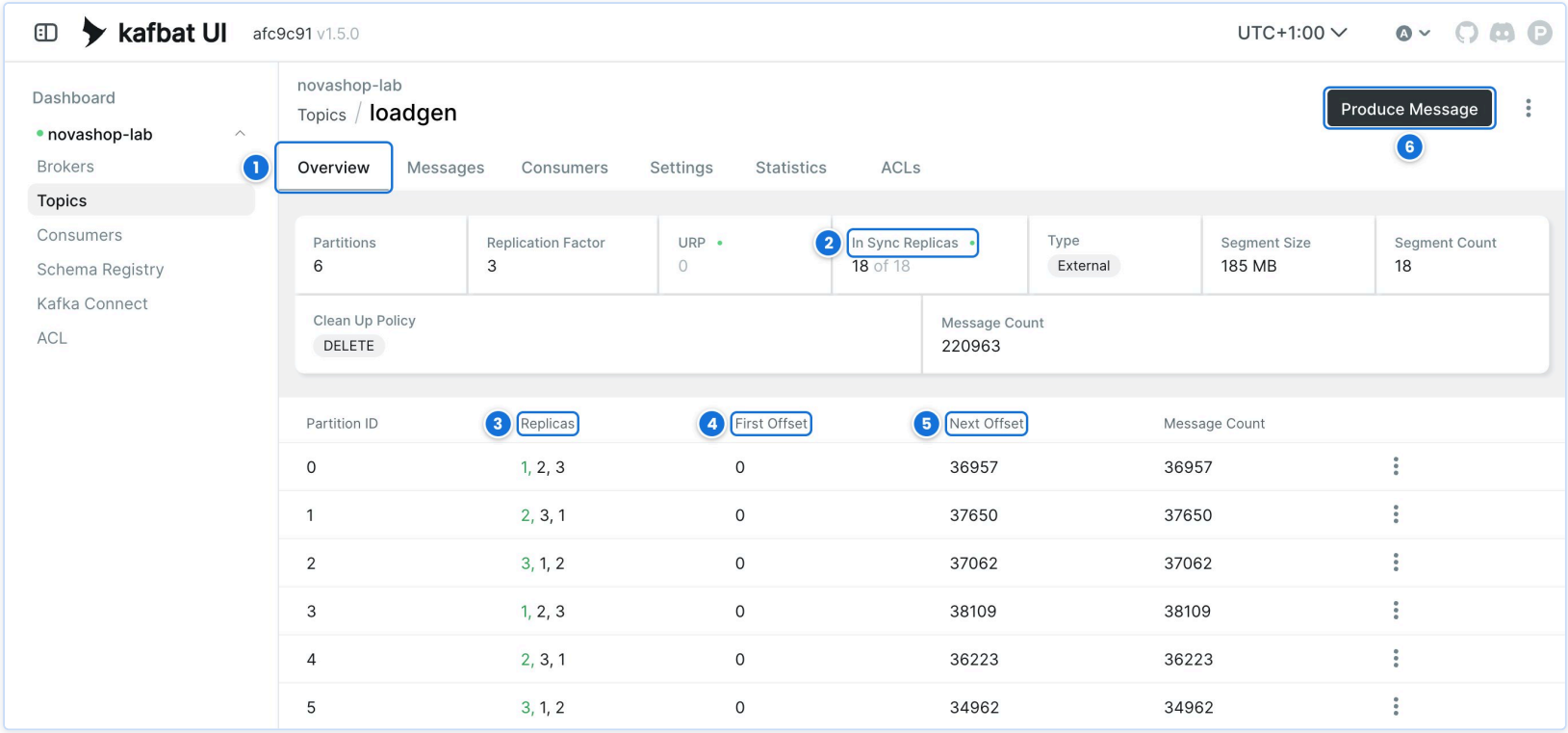
1. Nombre de partitions : le parallélisme maximal d'un consumer group. Il peut augmenter, jamais diminuer.
2. Politique de nettoyage : suppression par ancienneté ou par taille (*delete*), ou compaction par clé.
3. Facteur de réplication et nombre minimal de répliques synchronisées. Vides, ils prennent les valeurs du cluster : 3 répliques dans le lab.
4. Durée de rétention, avec des raccourcis de 1 heure à 4 semaines.
5. Toute autre configuration de topic, par exemple `segment.bytes` ou `retention.bytes`.

Lab ou production

La création automatique des topics est désactivée dans le lab, comme il est recommandé en production. En production, un topic se crée par un outil versionné (script, Terraform, opérateur Kubernetes) plutôt qu'à la souris.

Un topic : la vue d'ensemble

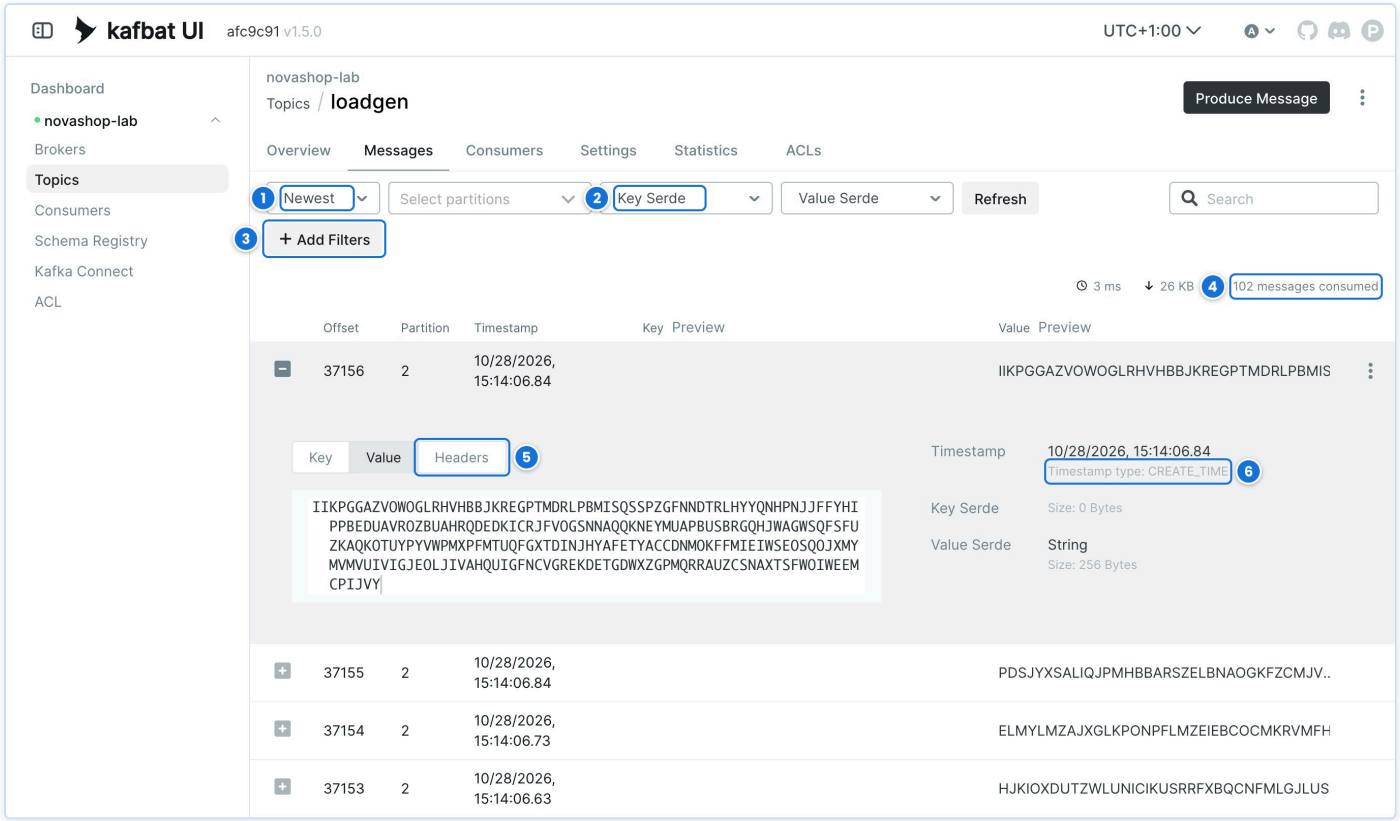
Le détail du topic `loadgen` : six onglets, et le placement de chaque partition sur les brokers.



1. Six onglets : Overview, Messages, Consumers, Settings, Statistics et ACLs.
2. 6 partitions fois 3 répliques : 18 répliques, toutes synchronisées. URP à 0.
3. Les brokers qui portent chaque partition, le leader en vert. Les leaders tournent d'un broker à l'autre.
4. Premier offset encore disponible. Il avance quand la rétention supprime les anciens segments.
5. Prochain offset qui sera écrit. Le nombre de messages est la différence entre les deux.
6. Écrire un message à la main. Le menu à trois points voisin permet de modifier, vider, recréer ou supprimer le topic.

Lire les messages

L'onglet Messages fait de Kafka UI un consommateur : il lit une centaine de messages à la position choisie et les décode.



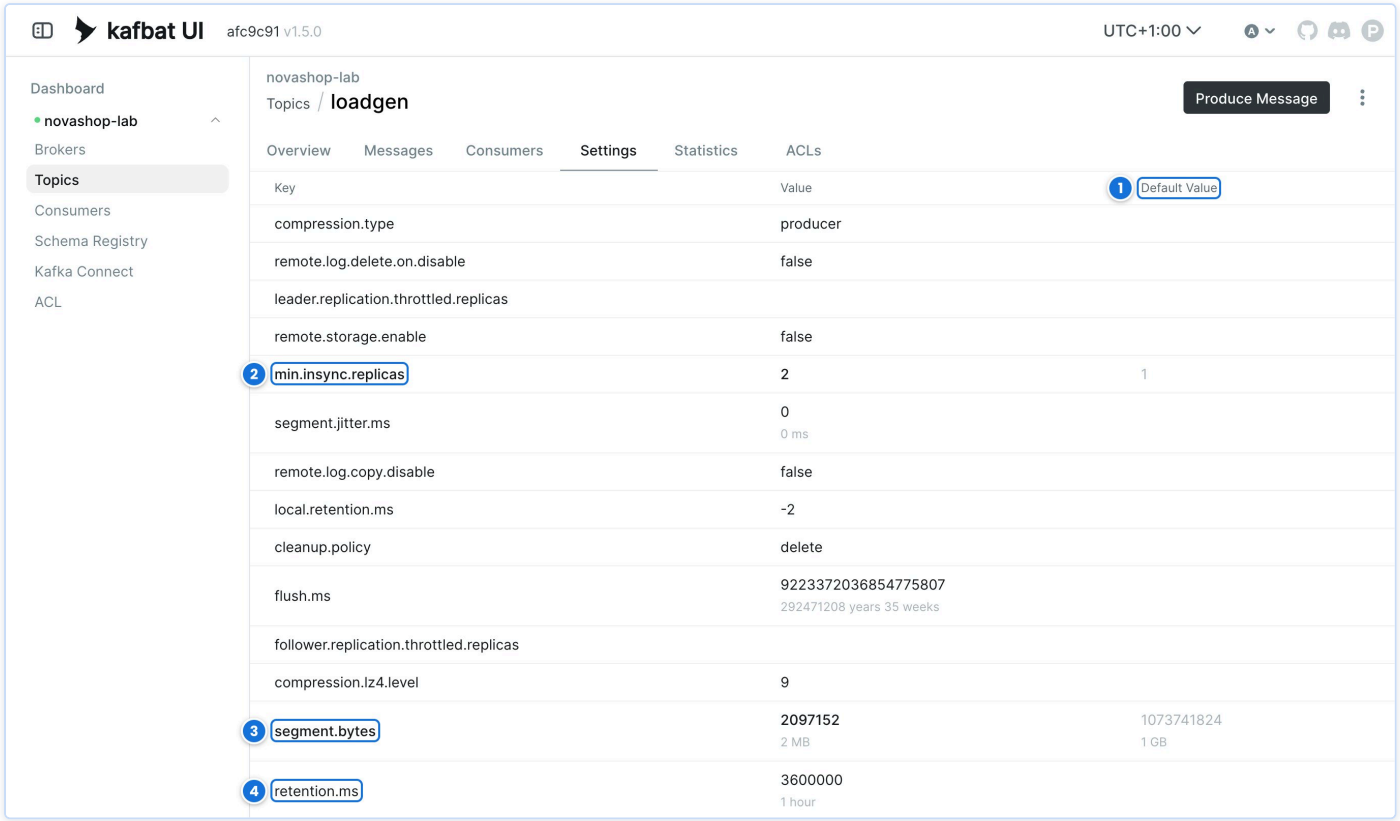
1. Position de lecture : les plus récents, les plus anciens, en direct, à partir d'un offset ou d'une date.
2. Décodage de la clé et de la valeur : texte, entiers, Avro via Schema Registry. Kafka ne stocke que des octets.
3. Filtres sur le contenu des messages.
4. Lecture sans consumer group : l'interface ne déplace la position d'aucune application.
5. Un message déplié : clé (absente ici), valeur et en-têtes.
6. Horodatage de type `CREATE_TIME` : fixé par le producteur, et non par le broker.

Lab ou production

Cet écran affiche les données métier, parfois personnelles. En production, la lecture des messages est réservée par rôle, ou les champs sensibles sont masqués.

Un topic : sa configuration

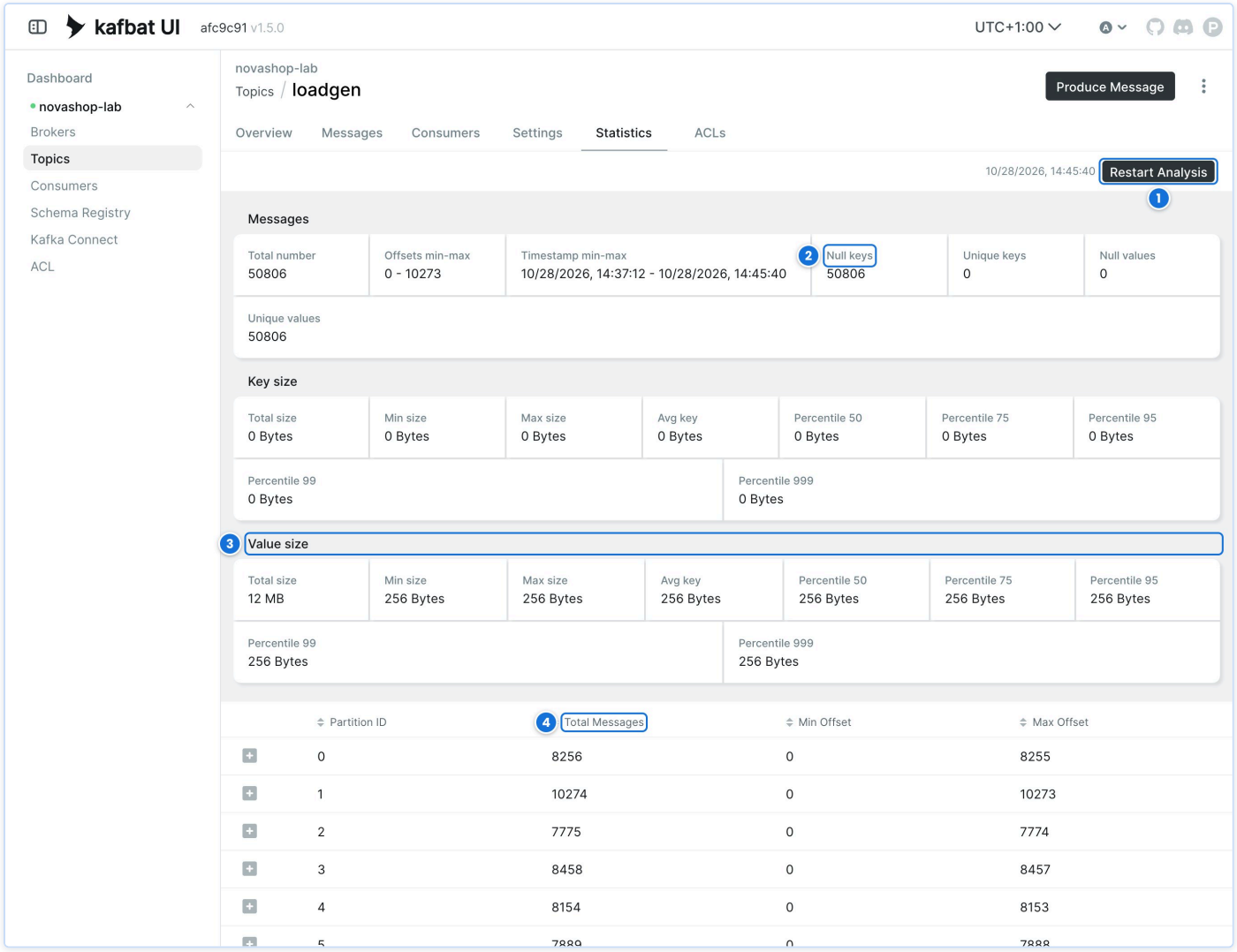
L'onglet Settings liste toutes les configurations du topic, l'équivalent de `kafka-configs.sh --describe`.



1. Valeur appliquée et valeur par défaut. Quand elles diffèrent, la valeur a été fixée pour ce topic.
2. 2 : un message envoyé en `acks=all` n'est acquitté qu'une fois écrit sur au moins 2 répliques.
3. Segments de 2 Mo au lieu de 1 Go. La rétention supprime des segments entiers : des segments petits la rendent visible en quelques minutes.
4. Rétention d'une heure, complétée par une limite de 10 Mo par partition : le générateur de charge ne remplit pas le disque.

Un topic : son profil

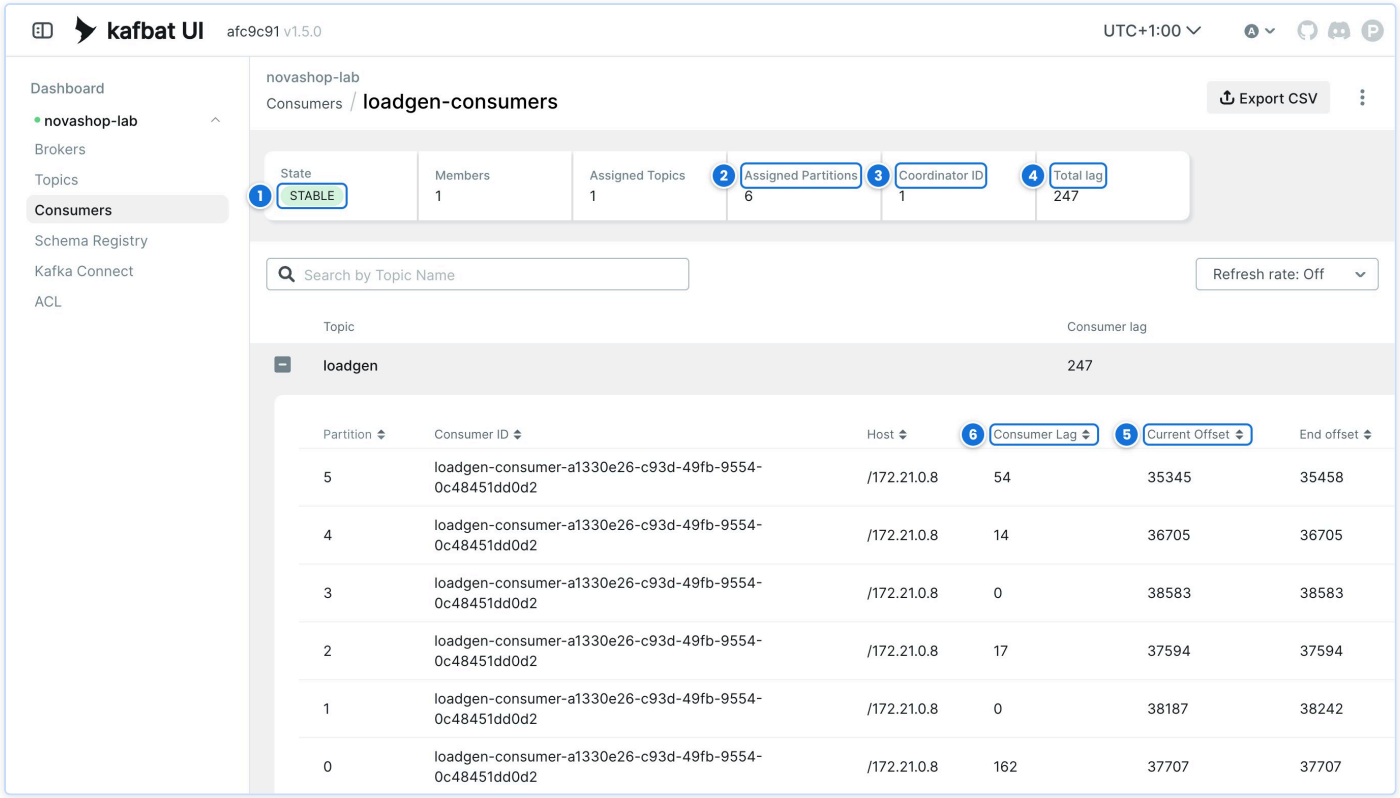
L'onglet Statistics relit le topic en entier et en tire un profil : clés, tailles, répartition par partition.



1. L'analyse relit tout le topic. Quelques secondes sur le lab ; sur un gros topic de production, à lancer hors des heures chargées. Avant la première analyse, un bandeau « 404 Not Found » s'affiche : il est sans conséquence.
2. Tous les messages sont sans clé : la partition est choisie par lot (partitionneur collant), et non par clé.
3. Taille des valeurs en percentiles : 256 octets pour tous les messages.
4. Le détail par partition : nombre de messages, offsets minimal et maximal.

Consumer groups

Pour chaque groupe : son état, ses membres, et le retard de lecture (lag) partition par partition.



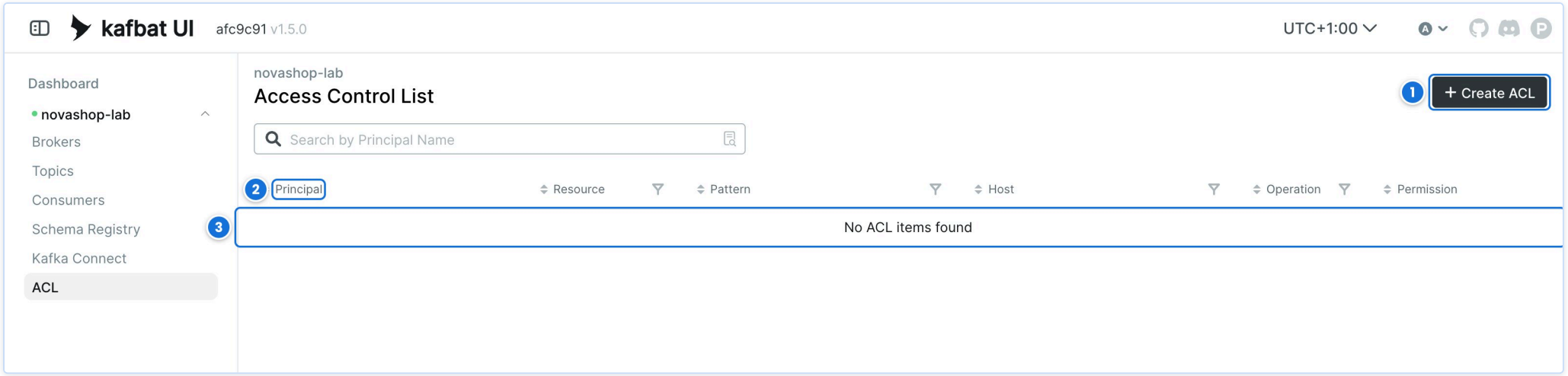
1. État du groupe. STABLE : chaque membre a ses partitions. EMPTY : aucun membre actif, les offsets sont conservés.
2. Un seul membre : il reçoit les 6 partitions du topic.
3. Le broker coordinateur du groupe : celui qui porte sa partition de `__consumer_offsets`.
4. Le retard total du groupe, somme des retards par partition.
5. Par partition : l'offset validé par le groupe (*Current Offset*) et la fin de la partition (*End offset*).
6. Leur différence : le lag. Il varie en permanence, car le consommateur valide sa position toutes les 5 secondes.

À savoir

Le menu à trois points du groupe propose *Reset offset* et *Delete consumer group*. La réinitialisation des offsets n'est possible que sur un groupe sans membre actif.

ACL

La rubrique ACL apparaît parce que l'autorisation est activée sur le cluster. Elle liste les règles d'accès et permet d'en créer.



1. Création d'une règle d'accès.

2. Une ACL dit qui (principal), sur quoi (ressource et motif), depuis où (hôte), pour quelle opération, et si elle est autorisée ou refusée.

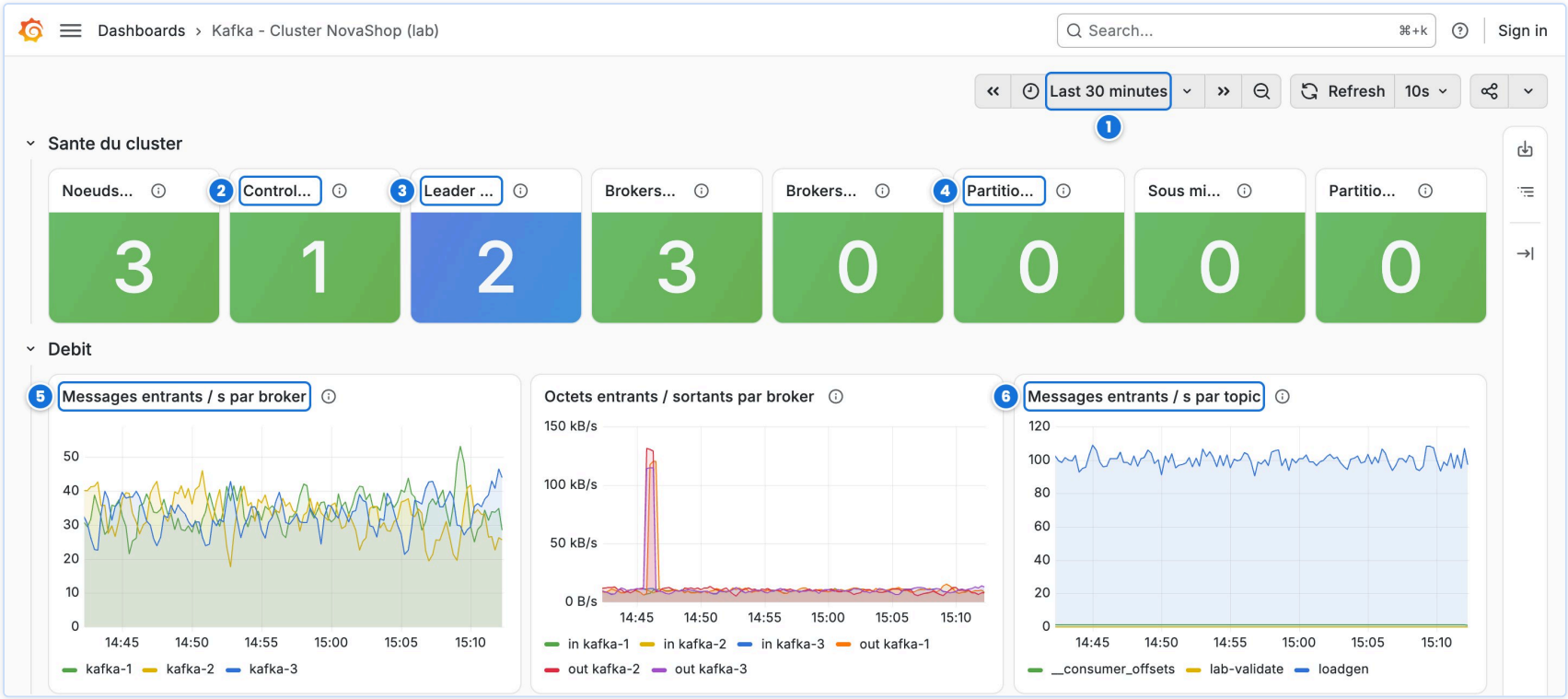
3. Aucune règle au démarrage du lab. Les comptes SCRAM et les ACL sont créés pendant la partie sécurité de la formation, et apparaissent alors ici.

Lab ou production

Lab : les clients non authentifiés (`User:ANONYMOUS`) sont super-utilisateurs, pour que les premiers travaux pratiques fonctionnent sans compte. Production : chaque application a son compte et ses ACL, et aucun accès anonyme n'est permis.

Santé du cluster et débit

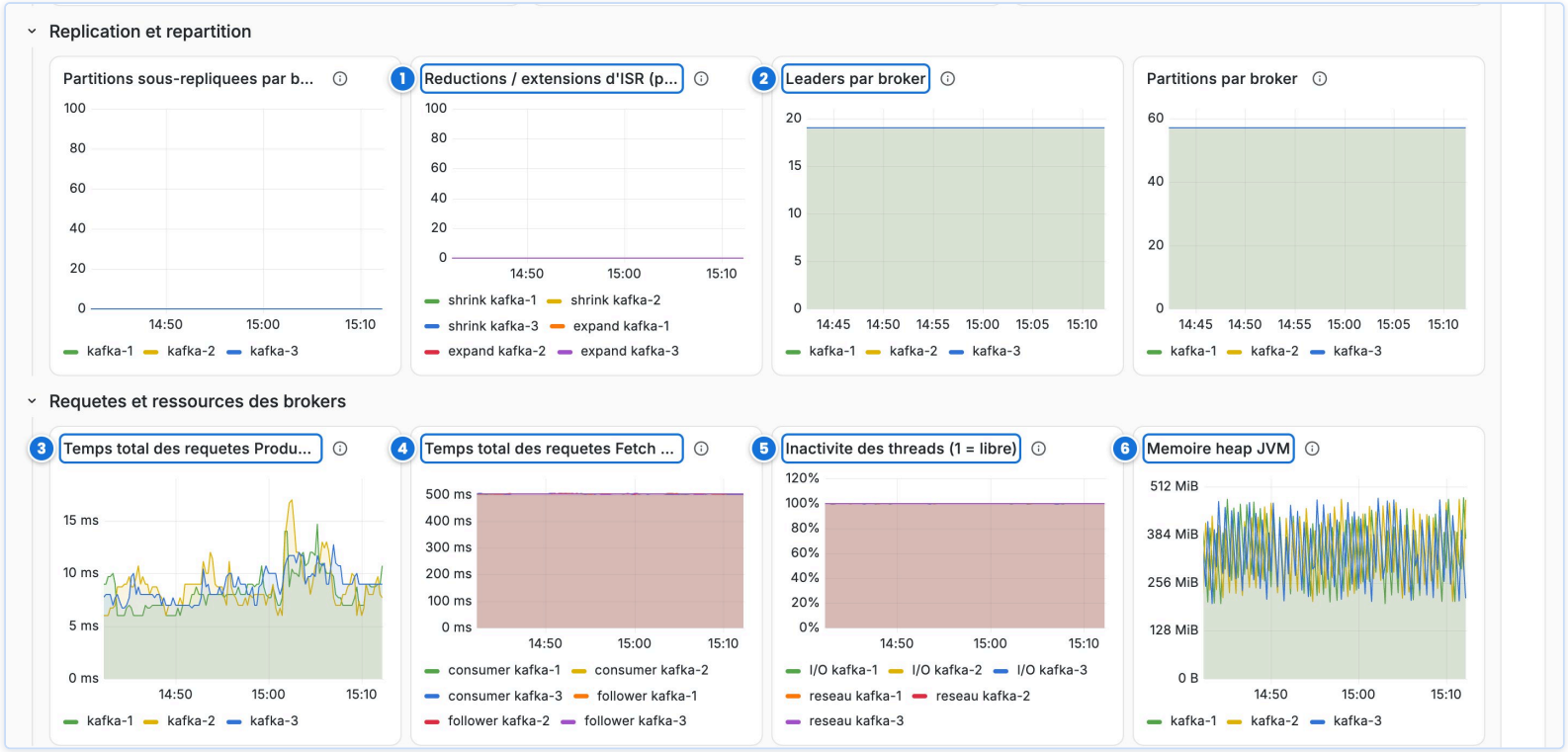
Le tableau de bord « Kafka - Cluster NovaShop (lab) » s'ouvre en page d'accueil. Sa première rangée forme le contrôle quotidien : 3 nœuds joignables, 1 contrôleur actif, 3 brokers actifs et des zéros partout ailleurs.



1. Période affichée et rafraîchissement automatique toutes les 10 s. En incident, réduire la période ; pour une analyse, l'élargir.
2. Exactement un contrôleur actif. Zéro : plus personne ne gère les métadonnées.
3. Numéro du nœud leader du quorum, en bleu : une information, pas un état de santé.
4. Trois niveaux de gravité : partitions sous-répliquées (une copie manque), sous min ISR (écritures `acks=all` refusées), offline (partition indisponible).
5. Débit des producteurs par broker, réplication exclue : environ un tiers des 100 messages par seconde chacun, car les leaders sont répartis.
6. Débit par topic : 100 messages par seconde pour `loadgen`.

Réplication et ressources des brokers

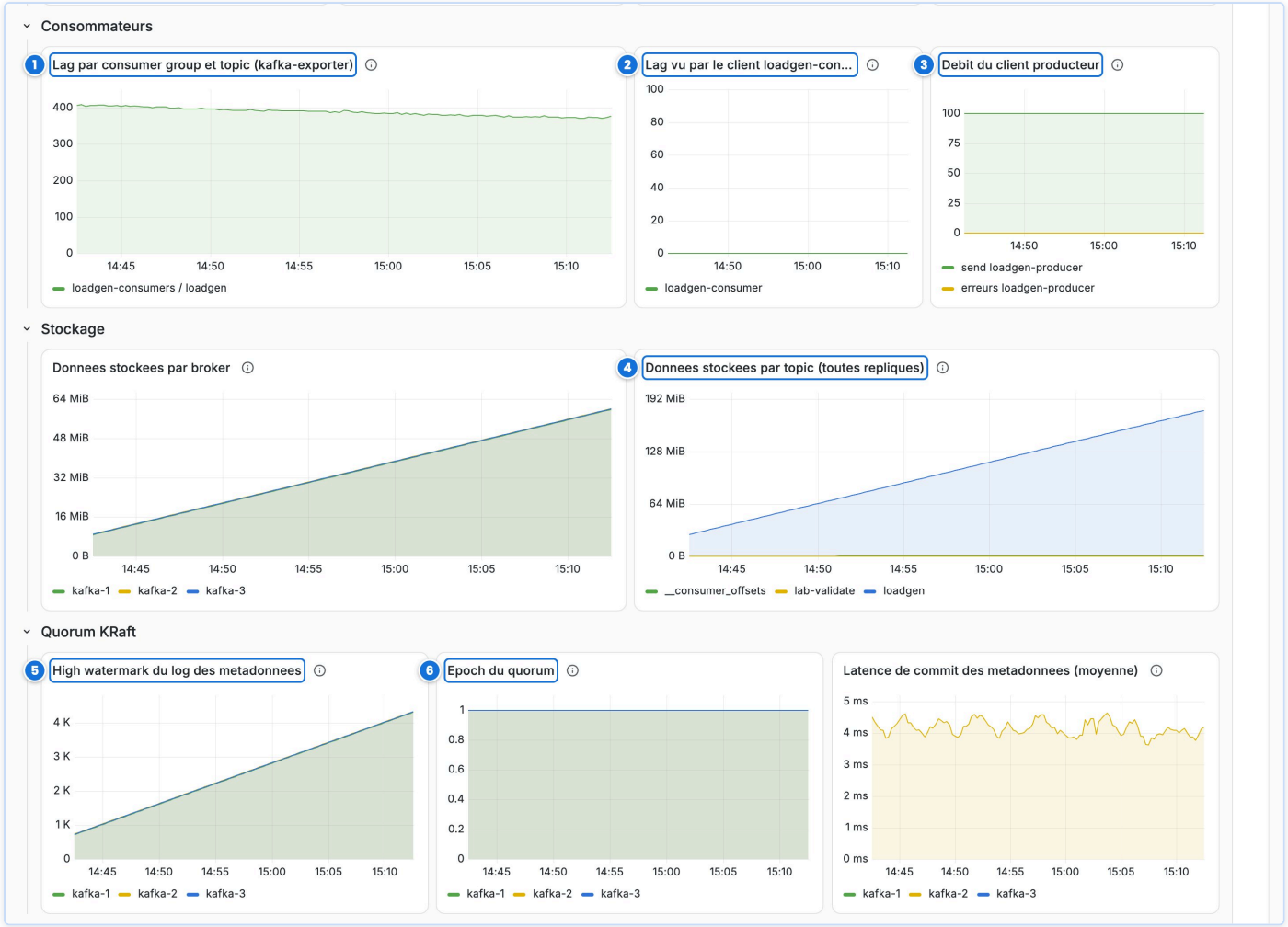
Ces deux rangées montrent si les copies suivent, si la charge est équilibrée, et si les brokers ont de la marge.



1. Doit rester à plat. Des réductions fréquentes signalent un follower lent : disque ou réseau.
2. 19 leaders par broker : la charge est équilibrée.
3. Temps de traitement d'une écriture au 99e percentile : environ 7 ms sur le lab.
4. Environ 500 ms, et c'est normal : un consommateur ou un follower attend jusqu'à 500 ms que des données arrivent (`fetch.max.wait.ms`, `replica.fetch.wait.max.ms`).
5. 1 : threads libres. Sous 0,3 de façon durable : broker saturé.
6. Dents de scie du ramasse-miettes, sous la limite de 512 Mo du lab. Kafka garde ses données dans le cache du système de fichiers, pas dans le heap.

Consommateurs, stockage et quorum

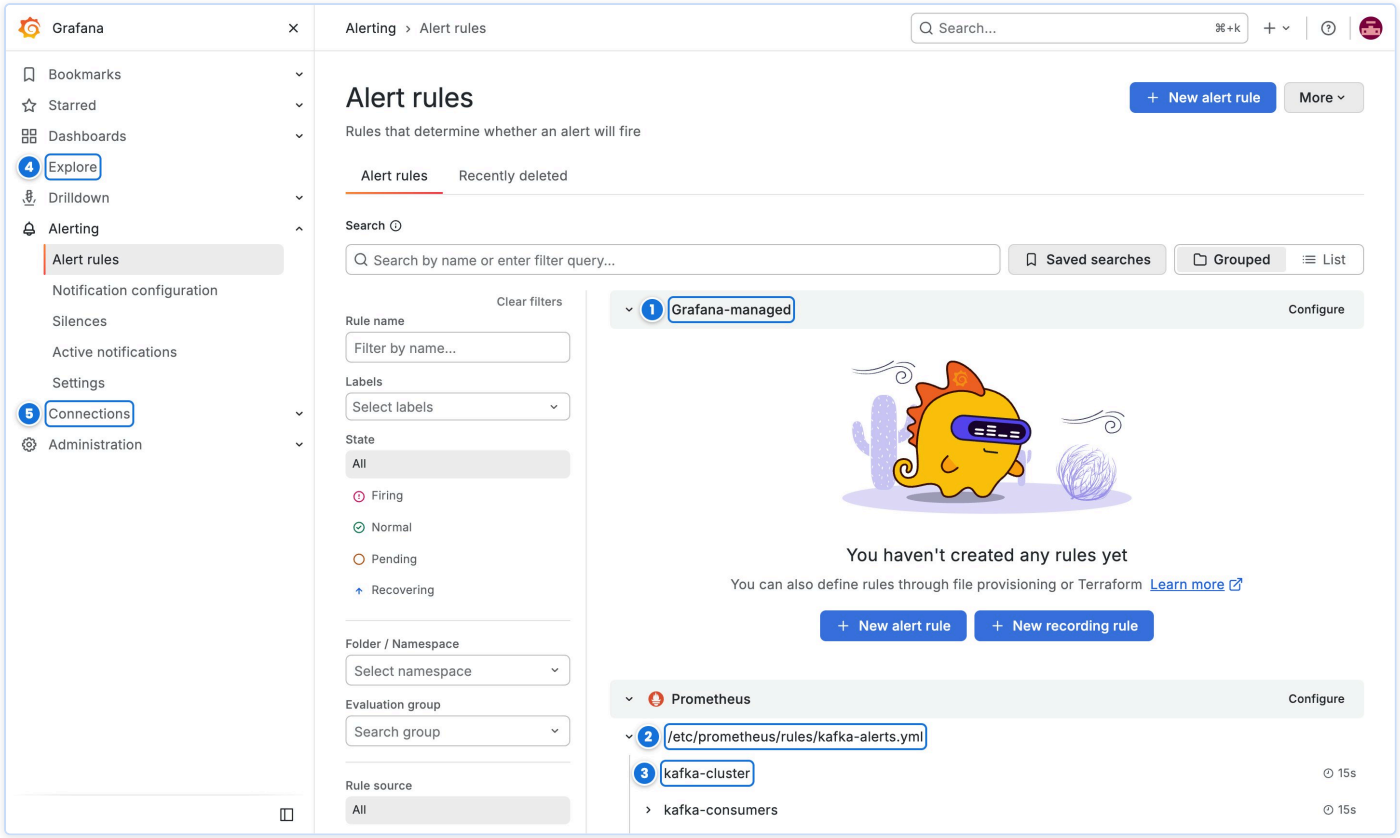
Les trois dernières rangées : le retard des applications, le volume sur disque, et l'état du quorum de contrôleurs KRaft.



1. Lag calculé côté cluster, à partir des offsets validés : environ 400 messages sur le lab.
2. Lag mesuré par le client, à partir de sa position de lecture : proche de 0. L'écart avec le précédent correspond aux messages lus mais pas encore validés. Pour alerter, on suit le lag côté cluster : il reste visible si le client s'arrête.
3. Envois par seconde et erreurs du producteur de charge.
4. Volume par topic, répliques comprises. Celui de `loadgen` monte jusqu'à sa limite de rétention (10 Mo par partition), puis plafonne. Une courbe qui monte sans jamais plafonner signale une rétention mal réglée.
5. Journal des métadonnées KRaft : il progresse en continu, et les courbes des trois contrôleurs restent superposées.
6. Nombre d'élections du leader du quorum : stable à 1 depuis le démarrage.

Alertes et sources de données

Grafana affiche les alertes, mais ne les définit pas : dans le lab, elles sont évaluées par Prometheus.



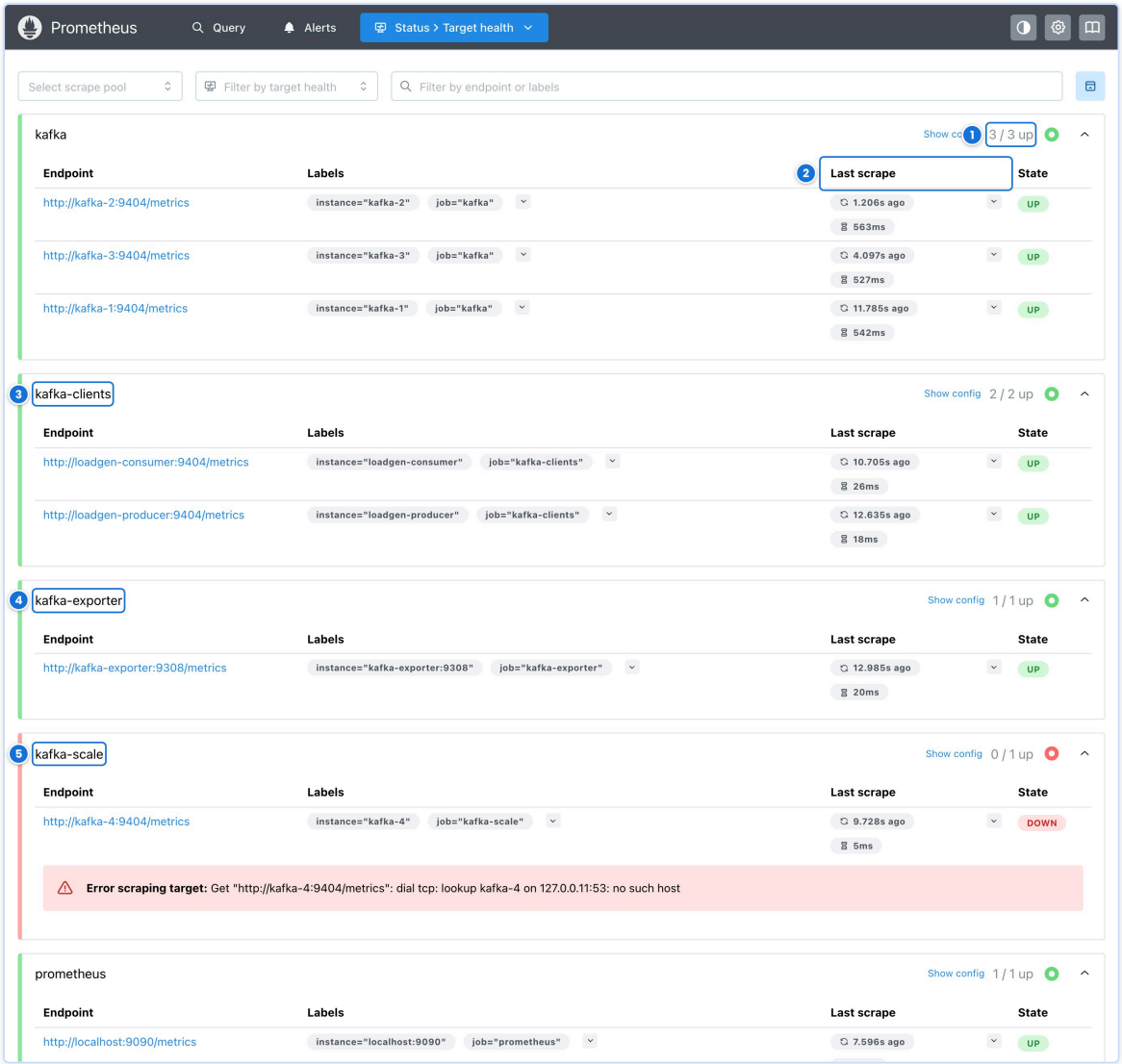
1. Aucune règle définie dans Grafana.
2. Les 9 règles du lab sont définies dans Prometheus. Grafana les affiche en lecture seule (*Data source-managed*).
3. Trois groupes de règles, évalués toutes les 15 s : cluster, consommateurs, stockage.
4. Explore : écrire et tester une requête PromQL sans créer de tableau de bord.
5. Une seule source de données, Prometheus. Grafana ne parle jamais directement à Kafka.

Lab ou production

Lab : consultation anonyme en lecture seule, compte `admin` au mot de passe connu.
Production : authentification d'entreprise et droits par équipe sur les dossiers de tableaux de bord.

La santé de la collecte

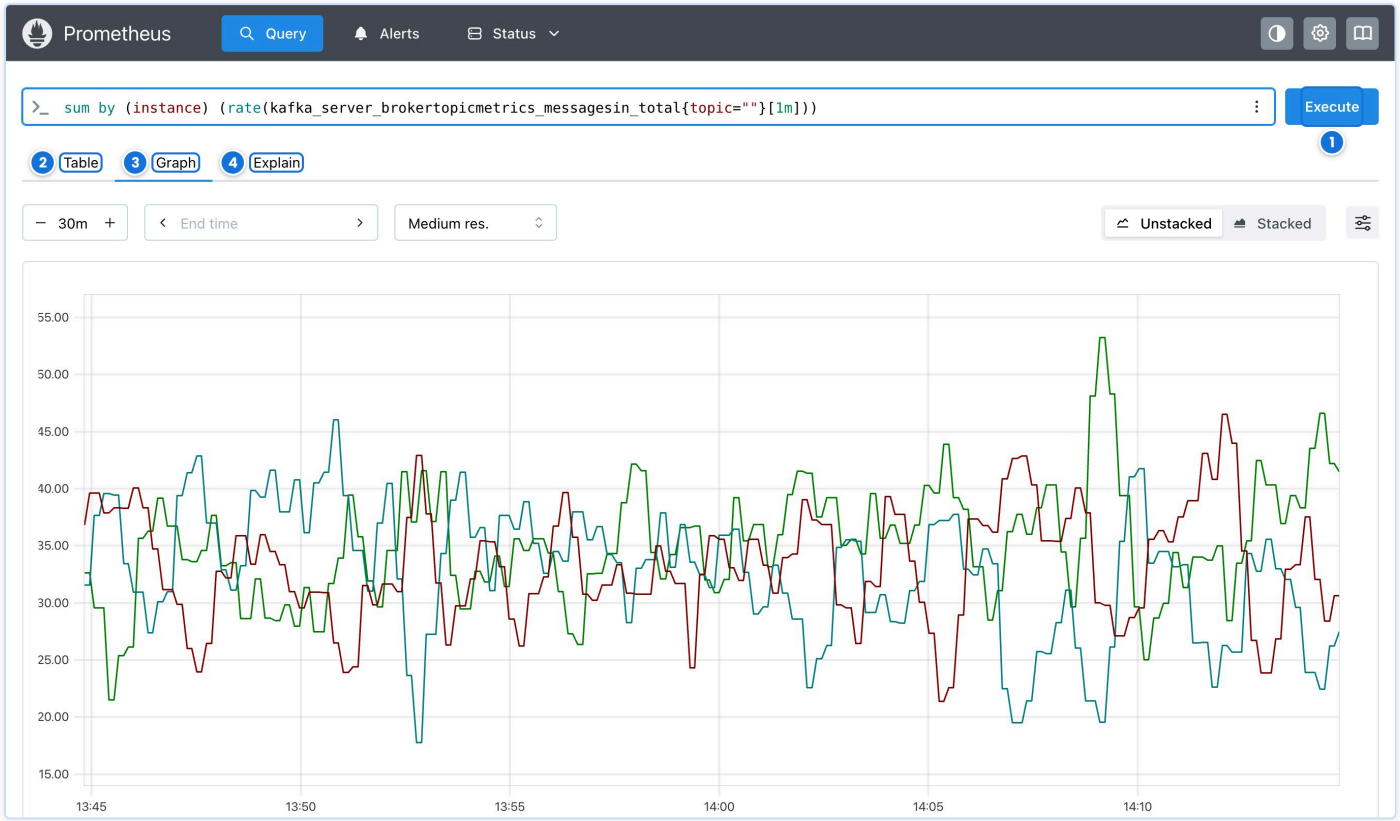
Prometheus interroge chaque cible toutes les 15 secondes. La page *Target health* (menu Status) est la première à ouvrir quand un tableau de bord est vide.



1. Les trois nœuds Kafka exposent leurs métriques par un agent JMX, sur le port 9404.
2. Âge et durée de la dernière collecte : environ une demi-seconde pour un broker, qui expose des milliers de séries.
3. Les deux clients du générateur de charge exposent aussi leurs métriques.
4. kafka-exporter calcule le lag des consumer groups côté cluster.
5. `kafka-4` n'existe pas encore : ce nœud est ajouté pendant les travaux pratiques d'exploitation. Cette cible en échec est attendue, et ne déclenche aucune alerte.

Interroger les métriques

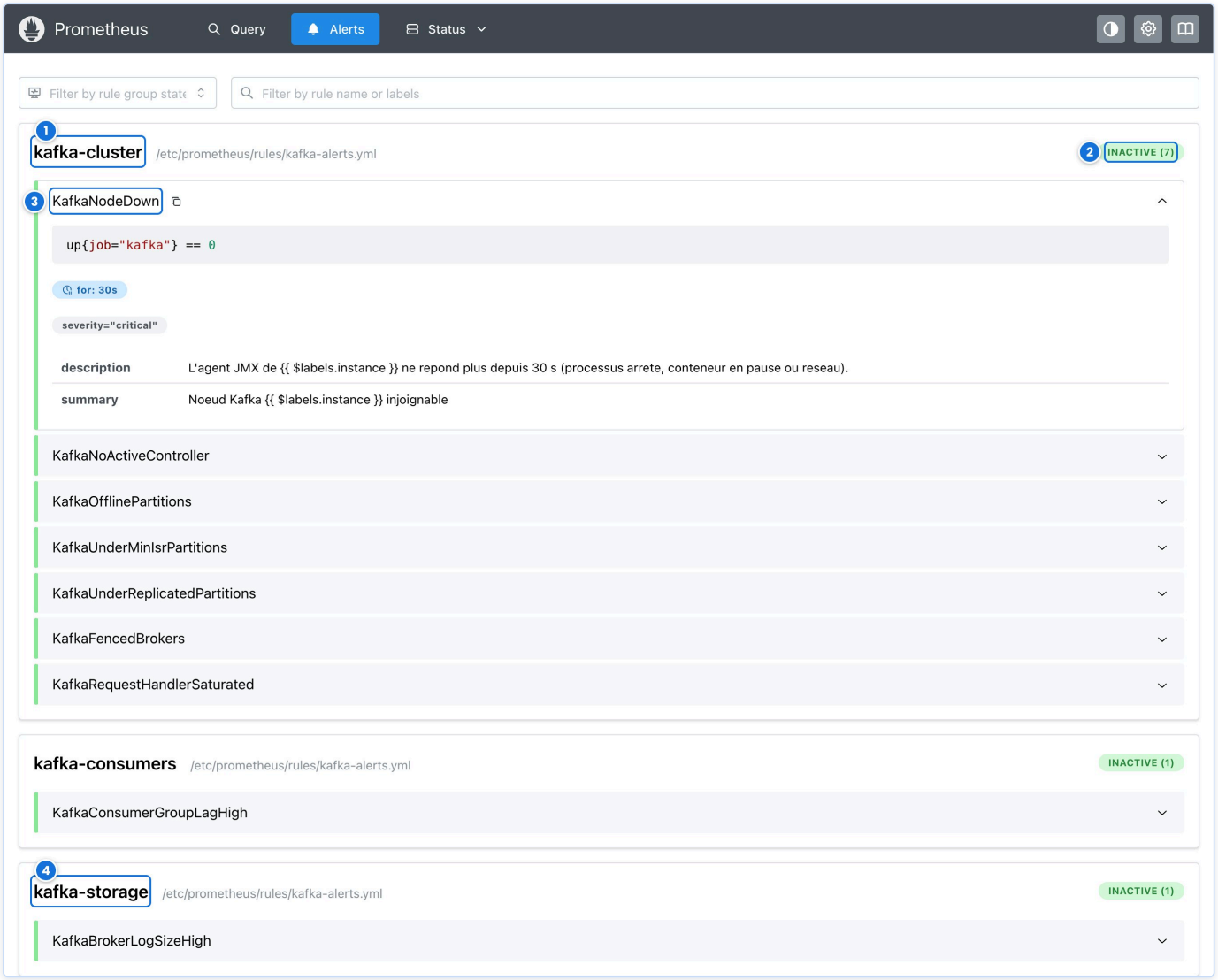
La page Query exécute du PromQL. Chaque panneau de Grafana est une requête de ce type ; ici, celle du panneau « Messages entrants / s par broker ».



1. La requête : un compteur de messages transformé en débit par seconde sur une minute (`rate`), puis additionné par broker (`sum by (instance)`).
2. Le résultat sous forme de tableau : une ligne par série, avec ses étiquettes. `up`, par exemple, vaut 1 pour chaque cible joignable.
3. Le résultat sous forme de courbe.
4. Décompose la requête et explique chaque fonction PromQL.
5. Une série par broker : environ 33 messages par seconde chacun.

Les règles d'alerte

Neuf règles, réparties en trois groupes. Toutes inactives : le cluster va bien.



1. Le groupe des règles du cluster, et le fichier qui les définit.
2. Trois états. Inactive : la condition est fausse. Pending : vraie depuis moins longtemps que la durée `for`. Firing : l'alerte est déclenchée.
3. Une règle dépliée : son expression PromQL (`up` égal à 0), sa durée `for` (30 s), sa sévérité et sa description.
4. Les groupes des consommateurs (lag) et du stockage (volume sur disque).

Lab ou production

Le lab n'a pas d'Alertmanager : les alertes se déclenchent mais ne sont notifiées à personne, et certains seuils sont abaissés pour être observables en travaux pratiques. En production, Alertmanager regroupe les alertes et les route vers l'astreinte.

Lab ou production : l'essentiel

Sujet	Dans le lab	En production
Topologie	3 nœuds combinés broker et contrôleur, sur un poste	Contrôleurs dédiés (3 ou 5), brokers sur des machines distinctes
Accès à Kafka UI	Sans authentification, toutes actions permises	SSO, rôles, lecture seule par défaut
Clients anonymes	Super-utilisateurs	Interdits : un compte et des ACL par application
Grafana	Consultation anonyme	Authentification d'entreprise, droits par équipe
Alertes	Visibles, non notifiées ; seuils abaissés	Alertmanager, routage vers l'astreinte ; seuils calés sur le volume réel
Rétention des métriques	3 jours	Selon les besoins d'historique, souvent avec un stockage déporté

Où ces écrans servent pendant la formation

- **Brokers et Topics** : déploiement et validation du cluster, puis gestion des topics en ligne de commande, contrôlée dans l'interface.
- **Consumer groups** : groupes, offsets et réinitialisation.
- **Configuration des brokers** : réglage de `min.insync.replicas` et panne de broker.
- **ACL** : sécurisation de l'accès (SCRAM, TLS, ACL, quotas).
- **Grafana et Prometheus** : observabilité, puis atelier de diagnostic d'incidents et TP final.
- **Schema Registry et Kafka Connect** : présentés en démonstration, lab en mode `full`. Ces rubriques ne figurent pas dans ce document.